COMP4221 Programming Assignment

Preparing and evaluating a structured campus FAQ assistant

Instructor: Yi R. (May) Fung | Fall 2026 | Total: 100 marks | Due date: To be announced

Build a small assistant that answers questions from supplied facts and returns the answer with its source ID, such as F001. You will clean structured records, implement nucleus probability filtering, select training targets, compute a smoothed loss, and run one short fine-tuning experiment.

The campus facts are fictional teaching examples, not HKUST policy. All data and runners are in this package. You do not need files or completed work from COMP4901B.

What you will complete

Part	Task	Marks
A	Normalize and validate FAQ records	15
B	Implement nucleus probability filtering	20
C	Final-response masking and balanced smoothed loss	30
D	One fine-tuning run and matched evaluation	35

Implement only the five TODO functions in student_tasks.py. The data builder, encoder, training loop, evaluator, plotting helper, and submission collector are provided. Part B is a separate probability exercise; Part D deliberately uses greedy decoding and does not call your nucleus function.

Budget approximately 8–12 hours of active work, plus installation and unattended compute. This is an estimate, not a measured guarantee. Only one 100-update experiment is required. An optional two-update smoke test is for debugging. No pretraining, model architecture implementation, extra dataset, sweep, or score threshold is required.

Start here

Read START_HERE.md for installation, the full command sequence, recovery, and troubleshooting. Use Python 3.10–3.12; a CUDA GPU is recommended for the experiment, but CPU is supported. Local function checks need no model download. Run commands from the extracted folder that contains student_tasks.py and raw_faq.jsonl.

```
python -m pip install -r requirements.txt
python preflight.py
# Implement Part A, then check just that part
python checks.py --part A
python prepare_data.py
```

You can check B, C1, and C2 separately while other TODOs remain unfinished. Run python checks.py after completing all five functions. An untouched starter is expected to report TODOs; that is not an installation failure.

Keep the supplied data, prompt builder, encoder, scoring rules, and experiment settings fixed. Implement your own solution for this assignment; any reuse or AI assistance must follow the published COMP4221 course policy.

Part A Normalize and validate FAQ records

15 marks | raw_faq.jsonl contains 120 records. Each accepted record must have string fields id, question, and answer. All paths below are relative to the extracted assignment folder.

A1 Implement two functions 10 marks

normalize_text(text) receives a string. In this order, decode HTML entities with html.unescape, apply Unicode NFKC normalization, then collapse every whitespace run to one ASCII space and trim the ends. Do not lowercase or remove punctuation. NFKC may itself change compatibility characters, including full-width punctuation; that change is required.

deduplicate_records(records) rejects non-dictionary entries and entries with missing or non-string required fields. Normalize all three fields, then reject IDs that do not exactly match uppercase F followed by three ASCII digits, or empty normalized questions or answers. Ignore extra fields. Return dictionaries containing only id, question, and answer.

For each valid key (normalized id, normalized question.casefold()), keep the first normalized record and preserve input order. Use casefold only for the comparison key; keep the normalized question's original case in the output. Different IDs are different keys even if their questions match.

A2 Report and interpret 5 marks

Run prepare_data.py. A correct implementation produces 100 retained facts, 300 training conversations, and 20 evaluation prompts. Report the raw count, valid count before deduplication, and unique count from data/counts.json. Show two actual before/after normalization examples with record IDs. Explain one risk when duplicate questions have conflicting answers and one way to detect it. No conflict-detector implementation is required.

Part B Nucleus probability filtering

20 marks | nucleus_distribution(logits, temperature, top_p) returns a probability vector in the original vocabulary order, not a sampled token ID.

B1 Implement filtering 15 marks

Input logits are finite, nonempty, one-dimensional floating-point values. Compute a stable softmax after dividing logits by temperature. Sort probabilities descending. Retain the smallest prefix with cumulative probability greater than or equal to top_p, including the crossing token. Zero the rest, renormalize, and restore the original vocabulary order.

At top_p=1 return the full softmax distribution. Keep at least one token. Reject nonfinite or nonpositive temperature and nonfinite top_p or top_p outside (0,1], using ValueError or TypeError. Any consistent ordering of ties is acceptable. Preserve device and shape; any floating output dtype is allowed. No gradient or sampling implementation is required.

B2 Work through an example 5 marks

For logits ln([0.50,0.30,0.15,0.05]) and temperature 1, give zero-based retained indices and the four-element output vector at top_p=0.70 and 0.90, rounded to three decimals. Explain crossing-token inclusion and why the retained count is not fixed. Include your B checker result. At an exact threshold equality, the prefix already reaches the threshold; no extra token is required.

Part C Final response supervision and smoothed loss

30 marks | The provided encoder returns token IDs and ordered, valid, nonoverlapping assistant spans. A span [start,end) includes token positions start through end−1, including that response's EOS token and excluding its role header. You do not need to infer spans or change a chat template.

C1 Supervise the last original response 15 marks

Complete final_response_features(input_ids, assistant_spans, max_length, pad_token_id). Inputs are Python lists of integer token IDs and span pairs; max_length is a positive integer. Return a dictionary with input_ids, labels, and attention_mask, each a one-dimensional torch.long tensor of length max_length.

Select the last assistant span in the original conversation. Copy input IDs into labels only at positions in this span; every other label is -100. Earlier assistant turns, system and user messages, and all role headers remain input context but are not supervised targets.

Right-truncate all arrays consistently, then right-pad. Real tokens have attention 1; padding has the supplied pad ID, attention 0, and label -100. Distinguish padding by position, even when pad and EOS have the same ID. If there is no assistant span, or the last original response is wholly truncated, all labels are -100. Never fall back to an earlier response.

In your report, construct a small example containing two assistant spans. Show input IDs, spans, labels, and attention with no truncation and with truncation that removes the final response. Explain the difference between useful context and supervised targets. The trainer skips examples with no valid shifted targets.

C2 Conversation balanced label smoothing 15 marks

Complete conversation_balanced_loss(logits, labels, epsilon=0.1). Logits have shape [batch, length, vocabulary]; labels are torch.long with shape [batch, length] on the same device. Labels are valid vocabulary indices or -100. Inputs are finite and batch, length, and vocabulary sizes are positive.

Align logits[:, :-1, :] with labels[:, 1:] for next-token prediction. Ignore targets equal to -100. For each valid target y, let log_p be stable log-softmax over the vocabulary. Its loss is (1−epsilon) × (−log_p[y]) + epsilon × mean_v(−log_p[v]). The uniform smoothing term includes the correct token.

Average valid token losses within each sequence, then average those means over nonempty sequences. Exclude sequences with no valid shifted targets. If none remain, including when length=1, return a differentiable scalar zero on the logits device. Preserve gradients. Reject epsilon outside [0,1] or nonfinite epsilon.

Derive the objective with tensor operations and log-softmax. Do not call CrossEntropyLoss or functional.cross_entropy in your implementation; the public checks may use them as an independent oracle. Never gather a vocabulary element using label -100.

Explain and verify

State the loss formula and the meaning of uniform smoothing. Give an arithmetic example with two conversations of unequal target lengths where conversation averaging and a global valid-token mean differ. Include C1 and C2 checker results. Public checks are representative; grading may use other inputs within these documented contracts.

Part D Fine tuning and grounded format evaluation

35 marks | Use HuggingFaceTB/SmolLM2-135M and the prepared FAQ data. Three training phrasings are provided per fact. The 20 evaluation prompts use a different question framing, but their facts appear in training and in the evaluation prompts. This tests responses to supplied evidence, not knowledge of unseen facts.

D1 Complete one run 15 marks

```
python train.py --steps 100 --output checkpoint
```

Complete 100 optimizer updates, using the final checkpoint. One update accumulates gradients from eight one-conversation micro-batches; it is not one example or one epoch. Fixed settings are seed 4221, sequence length 256, learning rate 2e-5, epsilon 0.1, and effective batch size 8. Smaller --steps values are for debugging, not a completed graded experiment.

Report actual hardware, completed updates, usable examples after truncation, initialization, and batch/length/learning-rate/smoothing settings from checkpoint/run_config.json. Plot checkpoint/loss.csv. Explain how the encoder and your C1 mask preserve earlier dialogue while supervising the final response. Do not tune the settings or select an intermediate checkpoint.

The runner prints progress and saves recovery state every 25 updates. START_HERE.md explains how to resume to a total of 100. If resources block completion, contact the course staff early and preserve logs; any alternative run length must follow a course-approved accommodation.

D2 Compare base and final models 15 marks

```
python evaluate.py --model HuggingFaceTB/SmolLM2-135M --output results/base
python evaluate.py --model checkpoint --output results/sft
python report_assets.py
```

Both evaluations must use the identical 20 prompts, provided encoding, greedy decoding, and 96-new-token cap. Part B is not used here. Each output folder receives summary.json and responses.json. The helper builds a loss plot and a browser-readable comparison; write your own interpretation.

A format pass requires two lines after trimming outer whitespace: the first starts exactly with Answer: followed by a space and nonempty text; the second matches Source: followed by a space and an ID such as F001. Internal extra lines, wrong label case, or commentary fail. Outer whitespace alone is ignored.

A grounded-format pass additionally requires the correct source ID and the expected answer after casefolding and whitespace collapse. Punctuation is significant; a paraphrase can fail. These are classroom checks, not IFEval. FAQ.md gives exact examples. Do not edit, postprocess, or hand-correct generated responses to raise scores.

Report both counts out of 20 and rates for both models. Show three paired raw responses, identifying their evaluation IDs; explain why each passes or fails and what changed. If an output is long, quote the relevant lines, label any omission, and keep the complete response in the ZIP.

D3 Interpret limitations 5 marks

Explain why lower training loss need not improve the two discrete rates. Identify one evaluation limitation and propose one follow-up without running it. No minimum score or improvement is required; poor model performance alone does not lose marks.

Submission and completion checklist

Prepare the report

Submit report.pdf, maximum 5 pages excluding references, and one code ZIP to Canvas. Include your name, student ID, and UST email. Concise tables and figures are sufficient; do not paste full code or all 20 outputs into the report.

Suggested allocation: A and B together about 1 page; C about 1 page; D configuration and loss about 1 page; D metrics and examples about 1–2 pages. Use REPORT_CHECKLIST.md to check completeness. No separate cover page or appendix is required.

Collect code and experiment evidence

```
python checks.py
python package_submission.py
```

The collector creates submission.zip and maps the two small files from checkpoint/ into experiment/. It deliberately excludes model weights, training_state.pt, model caches, and temporary outputs. Keep your checkpoint locally until grading is complete; you do not upload it. Upload report.pdf separately.

Required ZIP contents: student_tasks.py; all supplied Python runners and any helper code you changed or added; requirements.txt; raw_faq.jsonl; data/counts.json and data/clean_faq.json; experiment/run_config.json and experiment/loss.csv; results/base/summary.json and responses.json; and results/sft/summary.json and responses.json. Training and evaluation JSON files can be regenerated with prepare_data.py and need not be uploaded.

If you used different output folder names, pass --checkpoint, --base, and --sft to the collector and plotting helper. Neither helper completes TODOs or validates correctness. Inspect your ZIP before uploading and include any additional code dependencies you introduced.

How your work is assessed

Part A awards 10 marks for the two functions and 5 for counts, examples, and interpretation. Part B awards 15 for the implementation and 5 for the worked example, explanation, and checks. Part C awards 15 each for final-response supervision and the smoothed objective, including the requested explanations and validation.

Part D awards 15 for a reproducible completed run, configuration, loss plot, and encoding explanation; 15 for the matched evaluation, accurate metrics, and three interpreted pairs; and 5 for limitations and a follow-up proposal. Additional runs do not earn bonus marks. Public-check passes do not replace the report evidence or guarantee full implementation credit.

The same documented contracts apply to public and additional grading tests. You are not assessed on undocumented malformed inputs. Markers award independent criterion credit; a bug in one function does not automatically zero unrelated parts. Report incomplete work honestly and include any available evidence.

Final self check

Can you rerun the scripts from the extracted folder? Do all five TODOs have your implementation? Do the run configuration and loss log show 100 completed updates? Are both evaluations complete with the same 20 prompt IDs? Do your report's counts match the JSON? Does the ZIP contain the small experiment logs, with no model weights?

Work individually and follow published COMP4221 integrity, AI-use, and late-submission policies. Briefly disclose any permitted AI assistance and verify all code and reported results. Ask assignment questions through the course Canvas channel.